ĐẶC TẢ THIẾT KẾ BỘ GIẢI MÃ VITERBI

Module top: **`viterbi_decoder`** • Cấu hình: hard decision, K = 3, R = 1/2, khung 16 bit vào / 8 bit ra.

1. Tổng quan project

1.1 Mục đích

Đặc tả chức năng, kiến trúc và giao tiếp của bộ giải mã **`viterbi_decoder`**. Tài liệu là cơ sở chung cho thiết kế RTL, tích hợp, kiểm chứng và tổng hợp.

1.2 Phạm vi và cấu hình cơ sở

Project gồm bộ giải mã phần cứng với 5 block chính, mô hình tham chiếu, testbench, ràng buộc tổng hợp và báo cáo PPA. Thiết kế xử lý khung cố định 16 bit vào/8 bit ra, một khung tại một thời điểm, với một clock duy nhất.

1.3 Thuật ngữ và tài liệu tham khảo

Bảng 1. Thuật ngữ và tài liệu tham khảo

Thuật ngữ / Tài liệu	Ý nghĩa / Nội dung
BM (Branch Metric)	Chi phí nhánh: số bit khác nhau giữa cặp nhận và cặp mã của nhánh.
PM (Path Metric)	Chi phí đường đi: tổng BM tích lũy đến một trạng thái.
ACS (Add–Compare–Select)	Cộng, so sánh và chọn nhánh có chi phí nhỏ hơn.
Survivor	Nhánh được giữ lại sau phép **`ACS`**.
Traceback	Lần ngược theo các nhánh đã lưu để khôi phục bản tin.

2. Yêu cầu và thông số hệ thống

2.1 Thông số cố định

Bảng 2. Thông số cố định của bộ giải mã

Thông số	Quy định / Giá trị thiết kế
Kiểu giải mã	Quyết định cứng (Hard decision); dùng khoảng cách Hamming
Thông số mã chập	Constraint Length K = 3; Coding Rate R = 1/2
Đa thức sinh	$g_1(x) = x^2 + x + 1$ (111); $g_2(x) = x^2 + 1$ (101)

Thông số	Quy định / Giá trị thiết kế
Không gian trạng thái	2^{K-1} = 4 trạng thái: 00, 01, 10, 11 (quy ước s1s0, chuyển tiếp s_1s_0 -> us_1)
Kích thước khung dữ liệu	Đầu vào: 16 bit (data_in[15:0]); Đầu ra: 8 bit bản tin gốc (data_out[7:0]); 8 bước trellis
Điều kiện đầu / cuối	Bắt đầu tại state 00; kết thúc truy vết từ state có PM nhỏ nhất (ưu tiên hòa: 00 -> 01 -> 10 -> 11)
Độ rộng Metric & Bộ nhớ	Branch Metric (BM): 2 bit (0..2); Path Metric (PM): 5 bit (MAX_VAL = 31); Survivor Memory: 8 x 4 = 32 bit
Thời gian xử lý (Timing)	Độ trễ (Latency): 18 chu kỳ clock; chu kỳ nhận khung liên tiếp nhỏ nhất: 19 chu kỳ clock
Chỉ tiêu Tần số (Frequency)	Tối thiểu 200 MHz (Highest frequency win)
Chỉ tiêu Diện tích (Area)	Tối đa 7000 um^2
Chỉ tiêu Công suất (Power)	Tối đa 1.5 mW
Yêu cầu Kiểm chứng (Test)	Giải mã tự động toàn bộ chuỗi 16-bit từ input.txt, đối chiếu với output.txt, yêu cầu Pass 100%

2.2 Yêu cầu chức năng

Bảng 3. Yêu cầu chức năng

Mã	Yêu cầu	Hành vi bắt buộc
FR-01	Nhận khung	Chốt **`data_in`** khi IDLE và **`start`** = 1 tại cạnh lên **`clk`**.
FR-02	Giải mã	Dùng đúng trellis, thứ tự bit và quy tắc hòa metric.
FR-03	Hoàn thành	Mỗi khung được nhận tạo đúng một kết quả và một xung **`done`**, trừ khi bị hủy do reset hoặc FSM không hợp lệ.
FR-04	Xử lý khi bận	Bỏ qua **`start`** khi đang xử lý; giữ nguyên khung đã chốt.
FR-05	Giữ kết quả	Giữ **`data_out`** đến lần hoàn thành tiếp theo hoặc reset.

2.3 Phạm vi thuật toán

Mỗi khung bắt đầu độc lập từ state 00. Kết quả là đường hợp lệ có tổng khoảng cách Hamming nhỏ nhất; khi có nhiều đường tối ưu, áp dụng quy tắc hòa.

3. Thuật toán và quy ước dữ liệu

3.1 Trạng thái và thứ tự bit

State viết s_1s_0; s_1 là bit vào gần nhất. Với bit vào u: X_1 = u XOR s_1 XOR s_0, X_2 = u XOR s_0 và state tiếp theo = {u, s_1}. Xử lý **`data_in[15:14]`** trước, sau đó [13:12] đến [1:0]; bit cao của mỗi cặp là X_1. Bit đầu bản tin ở **`data_out[7]`**, bit cuối ở [0].

Bảng 4. Các bước xử lý, cặp bit nhận và vị trí bit giải mã

Bước trellis `t`	`fwd_step`	Cặp bit lấy từ `frame_reg`	Vị trí bit giải mã tương ứng
1	0	**`frame_reg[15:14]`**	**`data_out[7]`**
2	1	**`frame_reg[13:12]`**	**`data_out[6]`**
3	2	**`frame_reg[11:10]`**	**`data_out[5]`**
4	3	**`frame_reg[9:8]`**	**`data_out[4]`**
5	4	**`frame_reg[7:6]`**	**`data_out[3]`**
6	5	**`frame_reg[5:4]`**	**`data_out[2]`**
7	6	**`frame_reg[3:2]`**	**`data_out[1]`**
8	7	**`frame_reg[1:0]`**	**`data_out[0]`**

`frame_reg` lưu toàn bộ **`data_in[15:0]`** tại cạnh nhận khung. Trong FORWARD, các cặp bit được xử lý từ trái sang phải; bit cao của mỗi cặp tương ứng X_1, bit thấp tương ứng X_2. Cột vị trí bit giải mã biểu diễn ánh xạ dữ liệu, không phải thời điểm ghi đầu ra. Các bit được khôi phục trong TRACEBACK và toàn bộ **`data_out`** chỉ cập nhật tại DONE.

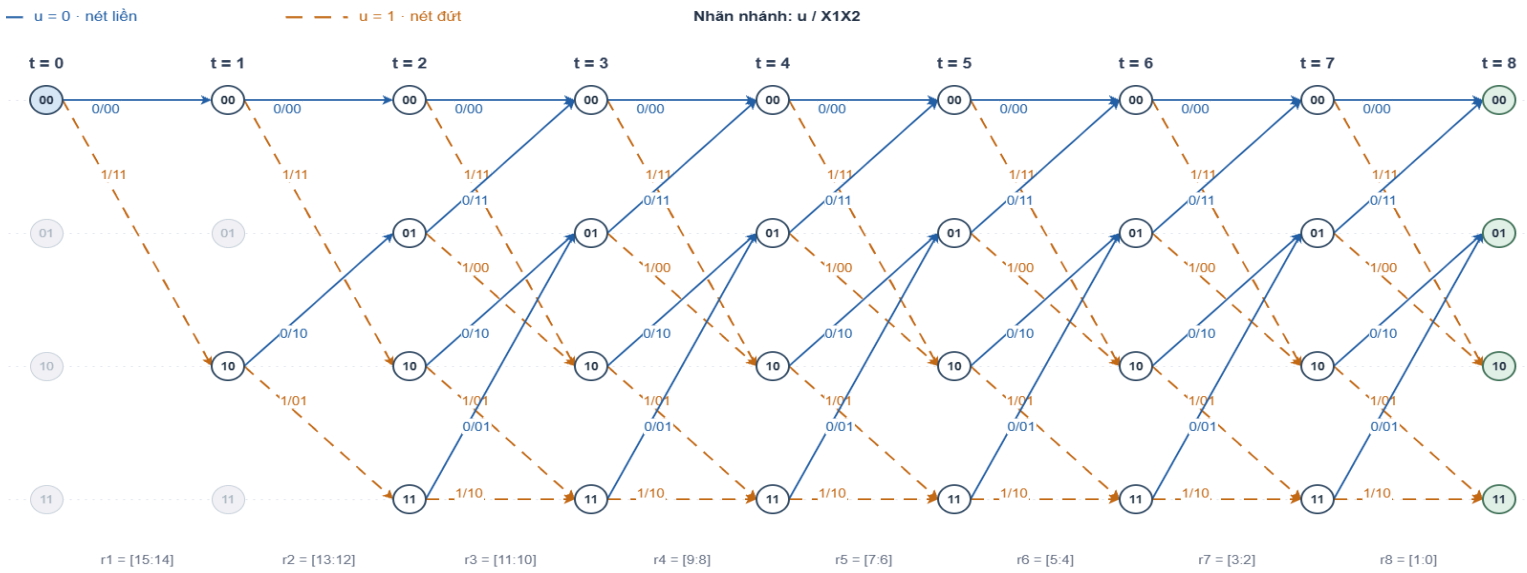
3.2 Trellis và ánh xạ nhánh

Mỗi trạng thái đích có hai nhánh vào. Cha A có bit thấp bằng 0, cha B có bit thấp bằng 1. Với trạng thái đích d_1d_0: cha A = {d0, 0}, cha B = {d_0, 1}; **`decision`** = 0 chọn A, **`decision`** = 1 chọn B.

Bảng 5. Ánh xạ trạng thái cha và cặp mã của nhánh

Trạng thái đích	Cha A / cặp mã	Cha B / cặp mã
00	00 / 00	01 / 11
01	10 / 10	11 / 01
10	00 / 11	01 / 00
11	10 / 01	11 / 10

Hình 1. Trellis bốn trạng thái qua tám bước xử lý



3.3 Tính metric và survivor

BM = (**`rx_pair[1]`** XOR X_1) + (**`rx_pair[0]`** XOR X_2), nhận 0–2. Khi nhận khung: **`PM00`** = 0, **`PM01`** = **`PM10`** = **`PM11`** = 31. Với mỗi nhánh, nếu PM cha = 31 thì ứng viên = 31; ngược lại ứng viên = min(31, PM cha + BM), cộng bằng 6 bit không dấu. Chọn ứng viên nhỏ hơn; nếu bằng nhau chọn A. Decision = 0 khi chọn A, = 1 khi chọn B, kể cả khi cả hai ứng viên bằng 31.

Bốn **`ACS`** tính từ cùng bộ PM cũ, sau đó bốn PM cập nhật đồng thời. Mỗi bước lưu **`decision_vec[3:0]`**, bit [s] ứng với state s. PM hợp lệ sau 8 bước không vượt 16. Cấu trúc BMC và ACS xem Hình 11 và 13, mục 9.2–9.3.

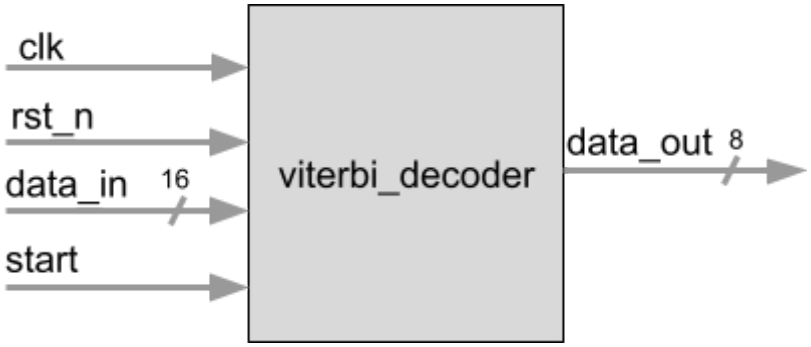
3.4 Chọn trạng thái cuối và truy vết

Chọn PM cuối nhỏ nhất; nếu hòa ưu tiên 00 → 01 → 10 → 11. Với **`tb_step`** từ 7 về 0: lấy **`tb_state[1]`** ghi **`decoded_work[7 - tb_step]`**, đọc **`decision`** = **`survivor_word[tb_state]`**, rồi cập nhật **`tb_state`** = {**`tb_state[0]`**, **`decision`**}. Bit kết quả và decision đều lấy từ state trước khi cập nhật. Không đảo lại **`decoded_work`**; ở DONE chuyển cả 8 bit sang **`data_out`**. Lưu đồ chi tiết xem Hình 18, mục 9.5.

4. Giao diện top và giao thức

4.1 Cổng của `viterbi_decoder`

Hình 2. Giao diện top module



Bảng 6. Giao diện module `viterbi_decoder`

Tín hiệu	Chiều	Bit	Ý nghĩa
`clk`	Vào	1	Clock cạnh lên
`rst_n`	Vào	1	Reset tích cực thấp
`start`	Vào	1	Yêu cầu một clock khi IDLE
`data_in`	Vào	16	Khung gồm 8 cặp bit
`data_out`	Ra	8	Kết quả hợp lệ khi **`done`** = 1
`done`	Ra	1	Báo hoàn thành một clock

4.2 Nhận khung và trả kết quả

Ở cạnh lên khi IDLE và **`start`** = 1, chốt toàn bộ **`data_in`**, khởi tạo metric và bắt đầu khung mới. Không có hàng đợi hoặc busy/ready bên ngoài. **`start`** trong FORWARD, SELECT, TRACEBACK và DONE bị bỏ qua. **`data_in`** chỉ cần ổn định quanh cạnh nhận khung.

`done` là đầu ra có thanh ghi, cao đúng một clock khi kết quả hoàn chỉnh được chốt. **`data_out`** giữ giá trị cũ trong khi giải mã. Nguồn phát **`start`** một clock; nếu giữ **`start`** đến khi trở lại IDLE, mức 1 được nhận như yêu cầu mới ở cạnh IDLE kế tiếp.

Bảng 7. Giao thức nhận khung và trả kết quả theo trạng thái FSM

Trạng thái trước cạnh clock	`start`	Hoạt động tại cạnh lên	`data_out` sau cạnh	`done` sau cạnh
IDLE	0	Không nhận khung mới; tiếp tục chờ	Giữ nguyên	0

IDLE	1	Chốt **data_in** vào **frame_reg**; khởi tạo PM; đặt **fwd_step** = 0; chuyển sang FORWARD	Giữ nguyên	0
FORWARD / SELECT / TRACEBACK	0 hoặc 1	Tiếp tục xử lý khung hiện tại; bỏ qua yêu cầu **start**, không chốt khung mới	Giữ nguyên	0
DONE	0 hoặc 1	Chuyển **decoded_work** sang **data_out**; phát **done**; chuyển về IDLE. Yêu cầu **start** tại cạnh này bị bỏ qua	Kết quả khung vừa hoàn thành	1

Bảng 7 áp dụng khi **rst_n** = 1 và FSM hợp lệ. Trạng thái được xét ngay trước cạnh lên clock; giá trị đầu ra được hiểu ngay sau cạnh. **data_in** chỉ cần ổn định quanh cạnh nhận khung và có thể thay đổi sau đó vì khung đã được lưu trong **frame_reg**. Không có hàng đợi yêu cầu.

Nếu giữ **start** = 1 đến cạnh xử lý IDLE kế tiếp, module nhận đó là yêu cầu cho một khung mới. Vì vậy, nguồn nên phát **start** trong một clock khi module đang chờ.

4.3 Clock, reset và hủy khung

Ngoài thao tác reset, tất cả thanh ghi cập nhật tại cạnh lên **clk**. **rst_n** tích cực thấp, khẳng định bất đồng bộ; phía cấp reset phải nhả đồng bộ với **clk** và đáp ứng recovery/removal. Reset ưu tiên mọi lệnh/enable: FSM về IDLE; **done**, **data_out**, **frame_reg**, **decoded_work**, **tb_state** và bộ đếm về 0; cả bốn PM về 31.

Reset giữa khung hủy khung và không phát **done** cho khung bị hủy. Chỉ xử lý lại khi nhận **start** hợp lệ sau reset. Mảng survivor không cần xóa; không ghi khi reset và chỉ được dùng sau khi đã ghi mới đủ 8 hàng.

Bảng 8. Giá trị tín hiệu và thanh ghi khi reset, nhận khung và phục hồi FSM

Tín hiệu / thanh ghi	**Reset: rst_n = 0**	**Nhận khung: IDLE, start = 1, rst_n = 1**	**FSM không hợp lệ: 101/110/111**
state	IDLE	FORWARD	IDLE
done	0	0	0
data_out	0	Giữ nguyên	Giữ kết quả trước đó
frame_reg	0	Chốt **data_in[15:0]**	Giữ nguyên

PM[0..3]	[31, 31, 31, 31]	[0, 31, 31, 31]	Giữ nguyên
fwd_step	0	0	0
tb_step	0	Giữ nguyên	0
tb_state	0	Giữ nguyên	Giữ nguyên
decoded_work	0	Giữ nguyên	Giữ nguyên
survivor_mem	Không xóa mảng; không ghi	Giữ nguyên; chưa ghi	Giữ nguyên; không ghi
Các enable	Tất cả bằng 0	**frame_load** = 1, **frame_init** = 1; các enable còn lại bằng 0	Tất cả bằng 0

Reset có ưu tiên cao nhất và được khẳng định bất đồng bộ. Cột nhận khung và phục hồi FSM mô tả giá trị ngay sau cạnh lên clock thực hiện thao tác tương ứng, khi **rst_n** = 1. Các enable trong Bảng 8 là giá trị tổ hợp trước cạnh, dùng để điều khiển thao tác tại cạnh lên; chúng không phải giá trị đầu ra sau cạnh.

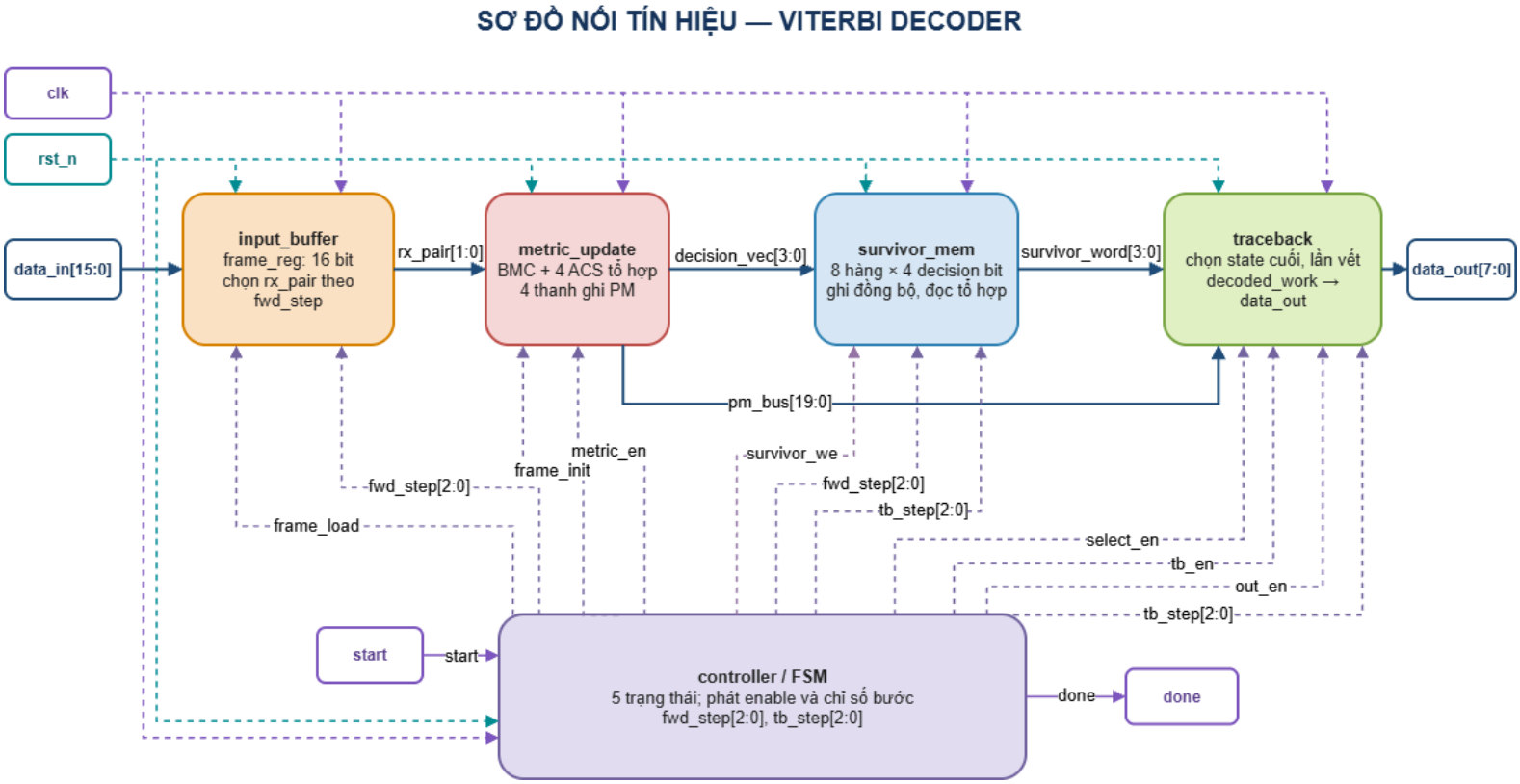
Khi nhận khung mới, chỉ khởi tạo những thành phần cần cho pha FORWARD. **tb_step** được đặt bằng 7, **tb_state** được chọn và **decoded_work** được xóa tại SELECT.

Mảng **survivor_mem** không cần reset vì mỗi khung ghi mới đủ tám hàng trước khi truy vết. Reset hoặc FSM không hợp lệ giữa quá trình xử lý sẽ hủy khung và không phát **done** cho khung đó.

5. Kiến trúc top và tổ chức phần cứng

5.1 Kết nối dữ liệu và điều khiển

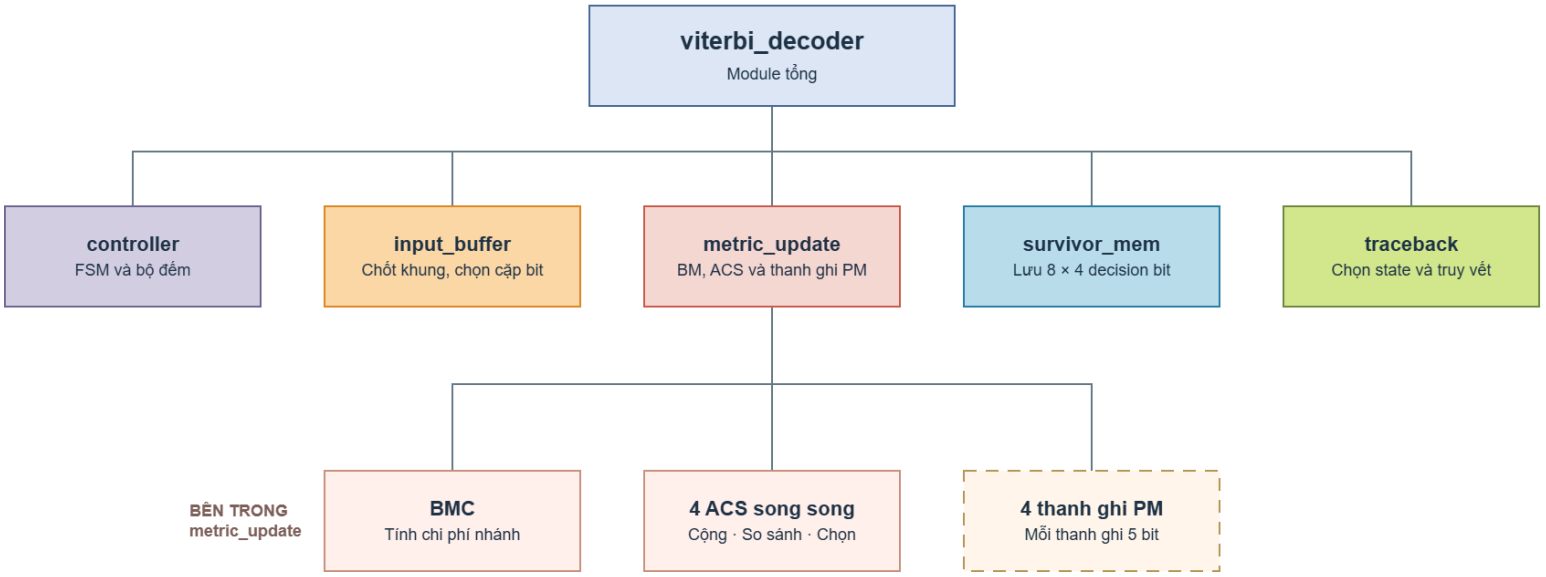
Hình 3. Sơ đồ kết nối dữ liệu và điều khiển



`data_in` → **`input_buffer`** → **`metric_update`** → **`survivor_mem`** → **`traceback`** → **`data_out`**. Bốn PM cuối đi trực tiếp từ **`metric_update`** tới **`traceback`**. Controller phát địa chỉ, enable và **`done`**; **`clk`**/**`rst_n`** cấp cho các block có lưu trữ.

5.2 Phân cấp module

Hình 4. Phân cấp module của bộ giải mã Viterbi



Bảng 9. Chức năng và cấu trúc các module

Module	Chức năng / cấu trúc
`viterbi_decoder`	Kết nối 5 block và các cổng top
`input_buffer`	Thanh ghi khung 16 bit; bộ chọn cặp nhận
`metric_update`	`BMC` + 4 `ACS` tổ hợp + 4 thanh ghi PM
`survivor_mem`	8 hàng × 4 decision; ghi đồng bộ, đọc tổ hợp
`traceback`	Chọn PM nhỏ nhất; lưu state, kết quả tạm và `data_out`
`controller`	FSM 5 trạng thái; `fwd_step`, `tb_step`; enable và `done`

5.3 Độ rộng và tài nguyên lưu trữ

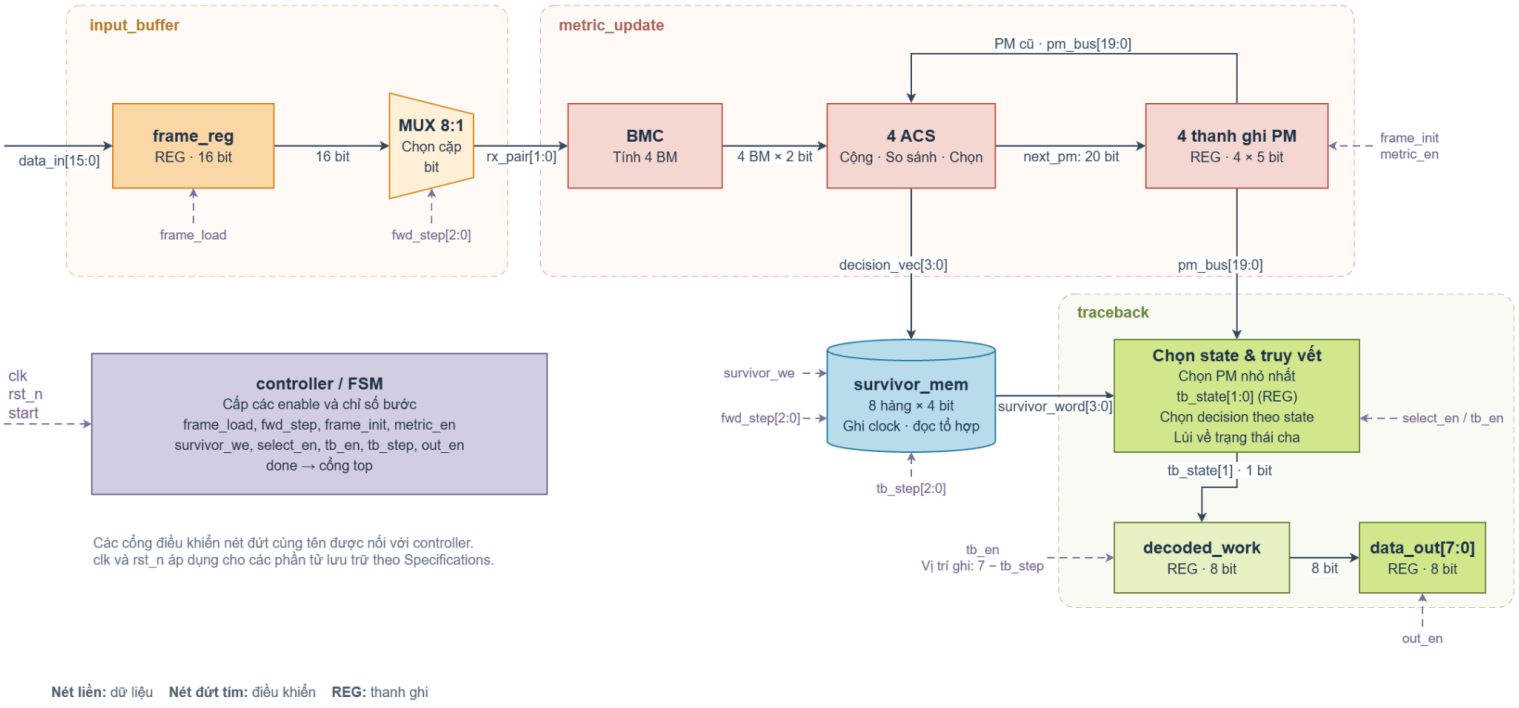
Bảng 10. Độ rộng dữ liệu và tài nguyên lưu trữ

Dữ liệu	Độ rộng
`frame_reg` / `rx_pair`	16 / 2 bit
BM / tổng ứng viên	2 / 6 bit không dấu
PM / `pm_bus`	4 × 5 / 20 bit
`decision_vec` / `survivor_mem`	4 / 8 × 4 bit
`fwd_step` / `tb_step` / `tb_state`	3 / 3 / 2 bit
`decoded_work` / `data_out`	8 / 8 bit
FSM	3 bit; mã hóa theo mục 7

6. Giao diện nội bộ và datapath

6.1 Tín hiệu dữ liệu

Hình 5. Datapath và các tín hiệu điều khiển nội bộ



Bảng 11. Tín hiệu dữ liệu giữa các module

Tín hiệu	Nguồn → Đích	Bit / ý nghĩa
`rx_pair`	`input_buffer` → `metric_update`	2; cặp nhận tại `fwd_step`
`bm00`, `bm01`, `bm10`, `bm11`	`BMC` → 4 `ACS`	Mỗi tín hiệu 2 bit
`pm_bus`	`metric_update` → `traceback`	20; 4 PM hiện tại
`decision_vec`	`metric_update` → `survivor_mem`	4; quyết định từ PM cũ và `rx_pair`
`survivor_word`	`survivor_mem` → `traceback`	4; hàng đọc tại `tb_step`
`fwd_step`	`controller` → `input_buffer`, `survivor_mem`	3; chỉ số chọn cặp và hàng ghi
`tb_step`	`controller` → `survivor_mem`, `traceback`	3; hàng đọc và vị trí bit kết quả

6.2 Tín hiệu điều khiển

Bảng 12. Tín hiệu điều khiển và chức năng

Tín hiệu	Đích	Điều kiện / thao tác
frame_load	**input_buffer**	IDLE và **start**; chốt **data_in**
frame_init	**metric_update**	IDLE và **start**; PM = (0,31,31,31)
metric_en	**metric_update**	FORWARD; cập nhật cả 4 PM
survivor_we	**survivor_mem**	FORWARD; ghi **decision_vec** tại **fwd_step**
select_en	**traceback**	SELECT; chọn state và xóa **decoded_work**
tb_en	**traceback**	TRACEBACK; thực hiện một bước truy vết
out_en	**traceback**	DONE; **data_out** nhận **decoded_work**

Các enable tích cực mức 1 và lấy mẫu tại cạnh lên; mặc định bằng 0, bao gồm lúc reset hoặc FSM không hợp lệ. Thanh ghi giữ giá trị khi không có lệnh. Reset ưu tiên cao nhất; **frame_init** ưu tiên **metric_en**.

Bảng 13. Ma trận trạng thái và bảy tín hiệu enable

Trạng thái	frame_load	frame_init	metric_en	survivor_we	select_en	tb_en	out_en
IDLE	**start**	**start**	0	0	0	0	0
FORWARD	0	0	1	1	0	0	0
SELECT	0	0	0	0	1	0	0
TRACEBACK	0	0	0	0	0	1	0
DONE	0	0	0	0	0	0	1
Reset (**rst_n** = 0)	0	0	0	0	0	0	0
FSM không hợp lệ	0	0	0	0	0	0	0

Giải thích:

IDLE: khi **start** = 1, vừa chốt khung bằng **frame_load**, vừa khởi tạo PM bằng **frame_init**. Khi **start** = 0, cả hai bằng 0.

FORWARD: bật đồng thời **metric_en** và **survivor_we** để cập nhật PM và lưu decision cùng một cạnh clock.

SELECT: bật **`select_en`** để chọn state bắt đầu truy vết và xóa kết quả tạm.

TRACEBACK: bật **`tb_en`** để thực hiện một bước truy vết.

DONE: bật **`out_en`** để chuyển kết quả tạm sang **`data_out`**.

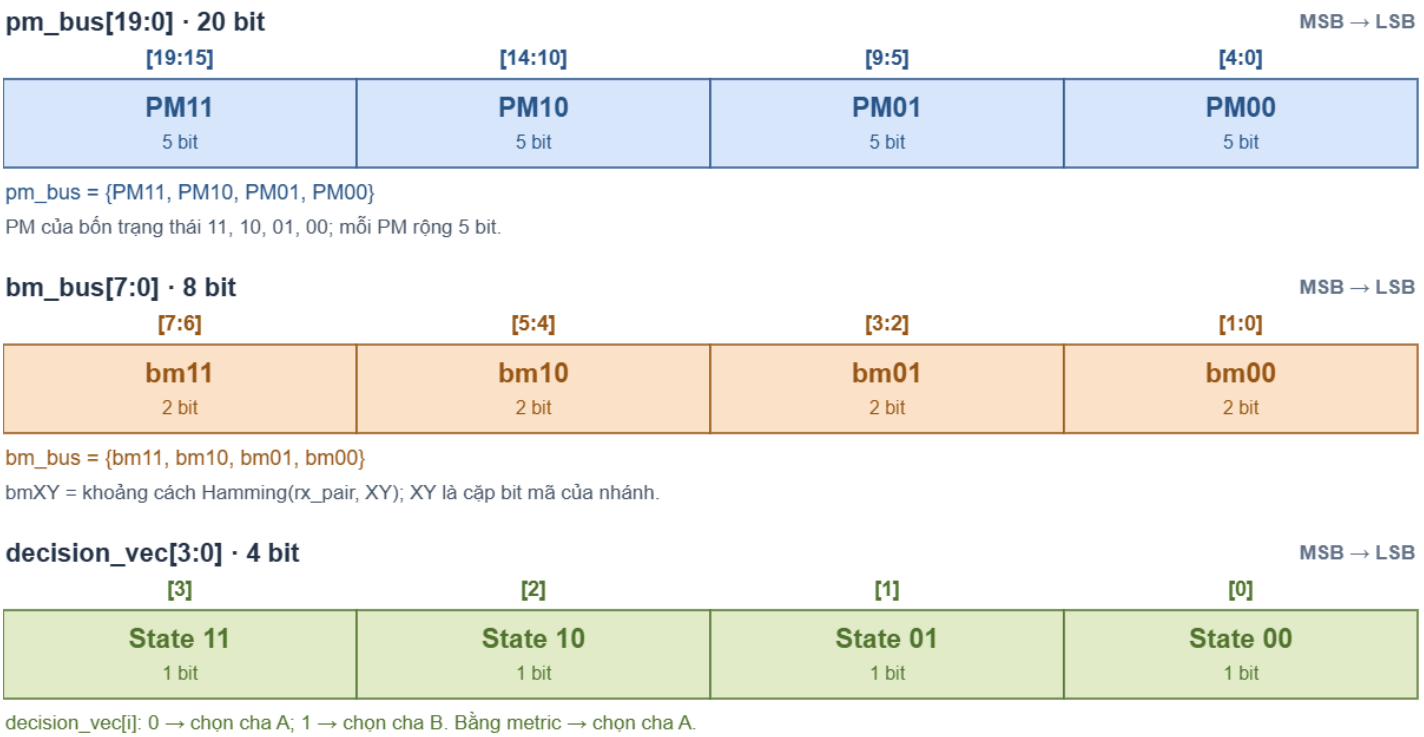
Giá trị 1: bật; 0: tắt; **`start`**: bằng giá trị tín hiệu **`start`**. Các hàng trạng thái hợp lệ áp dụng khi **`rst_n`** = 1. Enable được giải mã tổ hợp từ trạng thái trước cạnh clock và được lấy mẫu tại cạnh lên **`clk`**. Reset ưu tiên cao nhất và buộc toàn bộ enable về 0. FSM không hợp lệ gồm các mã 101, 110, 111.

6.3 Đóng gói bus và ánh xạ bộ nhớ

`pm_bus[4:0]` = **`PM00`**, [9:5] = **`PM01`**, [14:10] = **`PM10`**, [19:15] = **`PM11`**. Khi gộp BM: **`bm_bus`** = {**`bm11`**, **`bm10`**, **`bm01`**, **`bm00`**}. **`decision_vec[s]`** và **`survivor_word[s]`** ứng với state s = 0,1,2,3. **`survivor_mem[j]`** lưu decision của bước trellis t = j + 1, với j = 0…7. Tổ chức bộ nhớ và địa chỉ ghi, đọc xem Hình 16, mục 9.4.

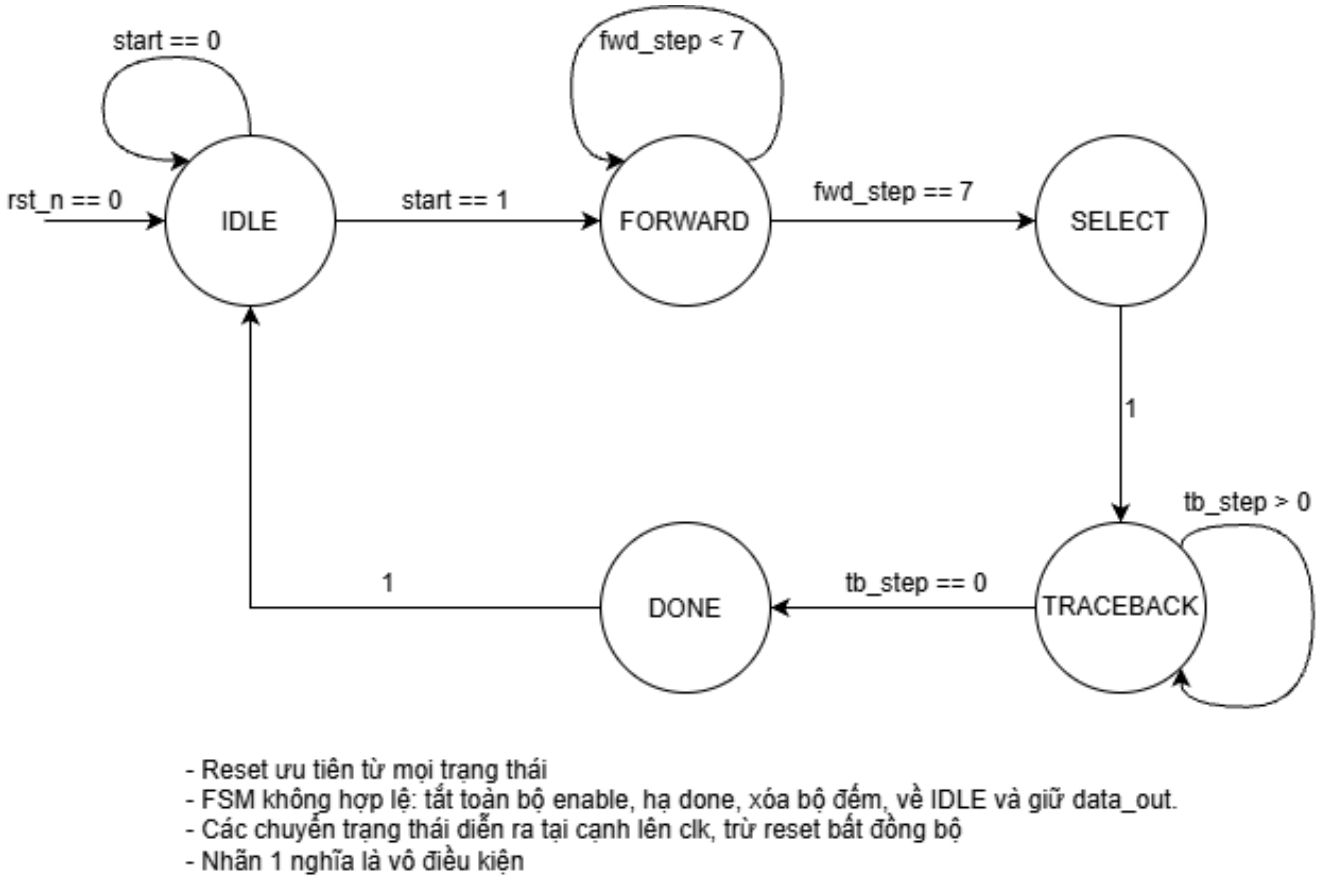
Ở FORWARD, **`decision_vec`** tổ hợp được tính từ PM trước cạnh clock; **`survivor_mem`** ghi nó cùng cạnh **`metric_update`** chốt PM mới. Sau cạnh ghi hàng 7, SELECT dùng các PM đã cập nhật. Không có đường phản hồi PM mới vào cùng bước **`ACS`**.

Hình 6. Đóng gói `pm_bus`, `bm_bus` và `decision_vec`



7. FSM và bảng điều khiển

Hình 7. Sơ đồ chuyển trạng thái FSM



7.1 Mã trạng thái và chuyển trạng thái

Bảng 14. Mã trạng thái và quy tắc chuyển trạng thái FSM

Trạng thái	Mã	Thao tác tại cạnh / chuyển trạng thái
IDLE	000	Hạ **`done`**. **`start`** = 1: **`frame_load`**, **`frame_init`**; **`fwd_step`** = 0; sang FORWARD. Nếu không, giữ IDLE.
FORWARD	001	**`metric_en`**, **`survivor_we`**. Nếu **`fwd_step`** < 7: tăng và giữ FORWARD; nếu = 7: sang SELECT.
SELECT	010	**`select_en`**; **`tb_step`** = 7; sang TRACEBACK.
TRACEBACK	011	**`tb_en`**. Nếu **`tb_step`** > 0: giảm và giữ TRACEBACK; nếu = 0: sang DONE.
DONE	100	**`out_en`**; đặt **`done`** = 1; sang IDLE. **`start`** tại cạnh này bị bỏ qua.

7.2 Bộ đếm, `done` và phục hồi

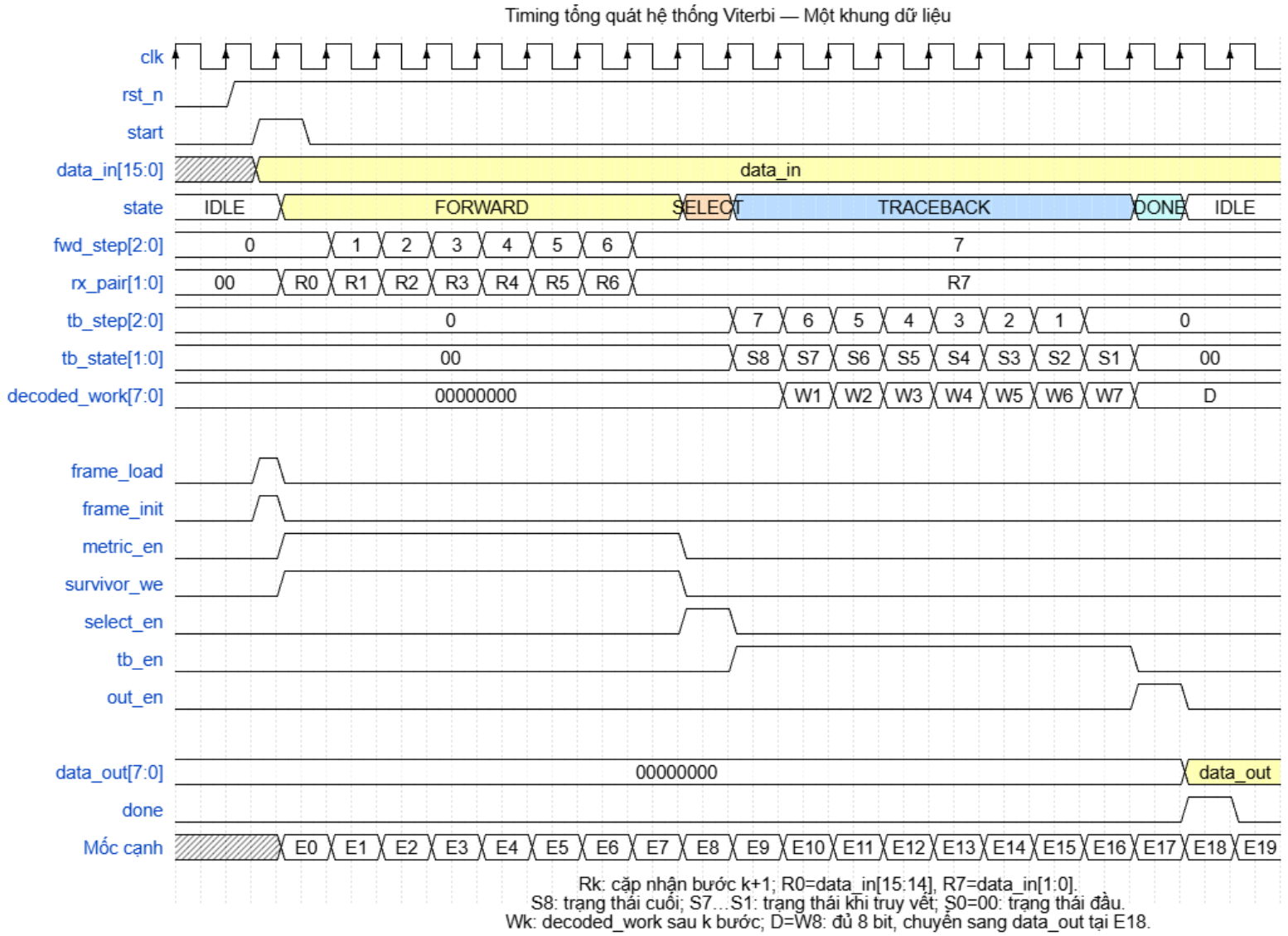
`fwd_step` chỉ tăng trong FORWARD, giữ tại 7 sau bước cuối; **`tb_step`** chỉ giảm trong TRACEBACK, giữ tại 0 sau bước cuối. Các bộ đếm không được tăng/giảm tràn. **`done`** được chốt theo trạng thái trước cạnh: chỉ cao sau cạnh xử lý DONE và hạ ở cạnh sau.

Với mã FSM 101/110/111: tắt toàn bộ enable, đưa về IDLE, hạ **`done`** và xóa bộ đếm; giữ **`data_out`** trước đó. Khung đang xử lý bị hủy; lần nhận tiếp theo khởi tạo lại PM và ghi mới đủ survivor.

Giá trị các tín hiệu và thanh ghi khi phục hồi FSM không hợp lệ được quy định tại Bảng 8, mục 4.3.

8. Thời gian xử lý và hiệu năng

Hình 8. Timing 1 khung



8.1 Bảng timing một khung

E0 là cạnh lên nhận **start** hợp lệ. Pha trong bảng là trạng thái ngay trước cạnh; kết quả được hiểu ngay sau cạnh. Survivor đọc tổ hợp, mỗi bước forward/**traceback** chiếm một clock.

Bảng 15. Thời gian xử lý một khung từ E0 đến E19

Cạnh	Pha	Hoạt động
E0	IDLE	Chốt khung, khởi tạo PM
E1–E8	FORWARD	8 cặp bit, 8 hàng survivor
E9	SELECT	Chọn state có PM nhỏ nhất
E10–E17	TRACEBACK	Ghi kết quả tạm [0]…[7]
E18	DONE	Chốt **data_out**; **done** = 1
E19	IDLE	Hạ **done**; có thể nhận khung mới

8.2 Quan hệ enable và dữ liệu theo clock

Waveform tổng quát dùng chung cho Bảng 15 và Bảng 16 được minh họa tại Hình 8.

Bảng 16. Enable và dữ liệu tại từng cạnh clock

Cạnh	Enable được lấy mẫu	Chỉ số / dữ liệu
E0	**frame_load**, **frame_init**	Chốt **data_in**; khởi tạo PM; **fwd_step** = 0
E1…E8	**metric_en**, **survivor_we**	**fwd_step** trước cạnh = 0…7
E9	**select_en**	Chốt **tb_state** từ PM cuối; **tb_step** = 7
E10…E17	**tb_en**	**tb_step** trước cạnh = 7…0; ghi **decoded_work[0]**…[7]
E18	**out_en**	**data_out** hợp lệ và **done** = 1
E19	**frame_load**/**frame_init** nếu **start** = 1	**done** = 0; có thể nhận khung kế tiếp

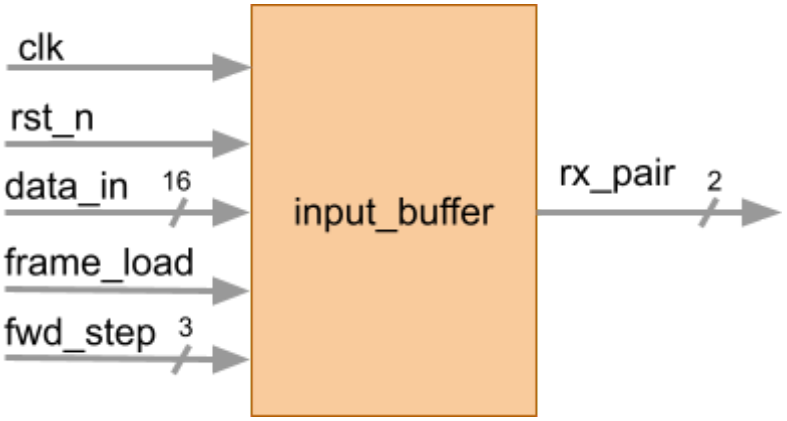
8.3 Chỉ tiêu hiệu năng

Độ trễ từ E0 đến E18 là đúng 18 chu kỳ; khoảng cách nhận khung nhỏ nhất 19 chu kỳ. Với fclk: thời gian giải mã = 18/fclk; thông lượng liên tục tối đa = 8 × fclk/19 bit dữ liệu/s. Không nhận khung tại E18 vì trước cạnh vẫn ở DONE. Đổi survivor sang đọc đồng bộ là thay đổi kiến trúc và phải sửa timing tương ứng.

9. Đặc tả chức năng các module

9.1 `input_buffer`

Hình 9. Giao diện module input_buffer



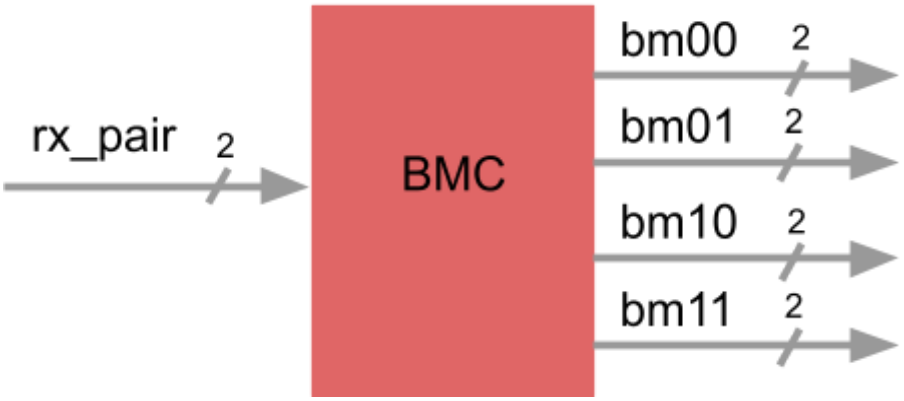
Bảng 17. Giao diện module `input_buffer`

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`clk`	Vào	1	Clock; chốt **`frame_reg`** tại cạnh lên.
`rst_n`	Vào	1	Reset bất đồng bộ, tích cực mức 0; xóa **`frame_reg`**.
`frame_load`	Vào	1	Cho phép nạp **`data_in`** vào **`frame_reg`**.
`data_in`	Vào	16	Khung mã hóa được chốt khi nhận **`start`** hợp lệ.
`fwd_step`	Vào	3	Chọn cặp bit nhận; chạy từ 0 đến 7.
`rx_pair`	Ra	2	Cặp bit tại bước hiện tại; đầu ra tổ hợp.

Reset xóa **`frame_reg`**. Tại **`frame_load`**, nạp **`data_in`**; ngoài ra giữ nguyên. **`rx_pair`** là đầu ra tổ hợp chọn theo **`fwd_step`**; miền chỉ số hợp lệ 0…7. **`fwd_step = 0`** chọn **`frame_reg[15:14]`**, tiếp tục cho đến **`fwd_step = 7`** chọn **`frame_reg[1:0]`**.

9.2 BMC — Branch Metric Computation

Hình 10. Giao diện module bmc



Bảng 18. Giao diện module BMC

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
rx_pair	Vào	2	Cặp bit nhận cần tính Branch Metric.
bm00	Ra	2	Khoảng cách Hamming giữa **rx_pair** và cặp mã 00.
bm01	Ra	2	Khoảng cách Hamming giữa **rx_pair** và cặp mã 01.
bm10	Ra	2	Khoảng cách Hamming giữa **rx_pair** và cặp mã 10.
bm11	Ra	2	Khoảng cách Hamming giữa **rx_pair** và cặp mã 11.

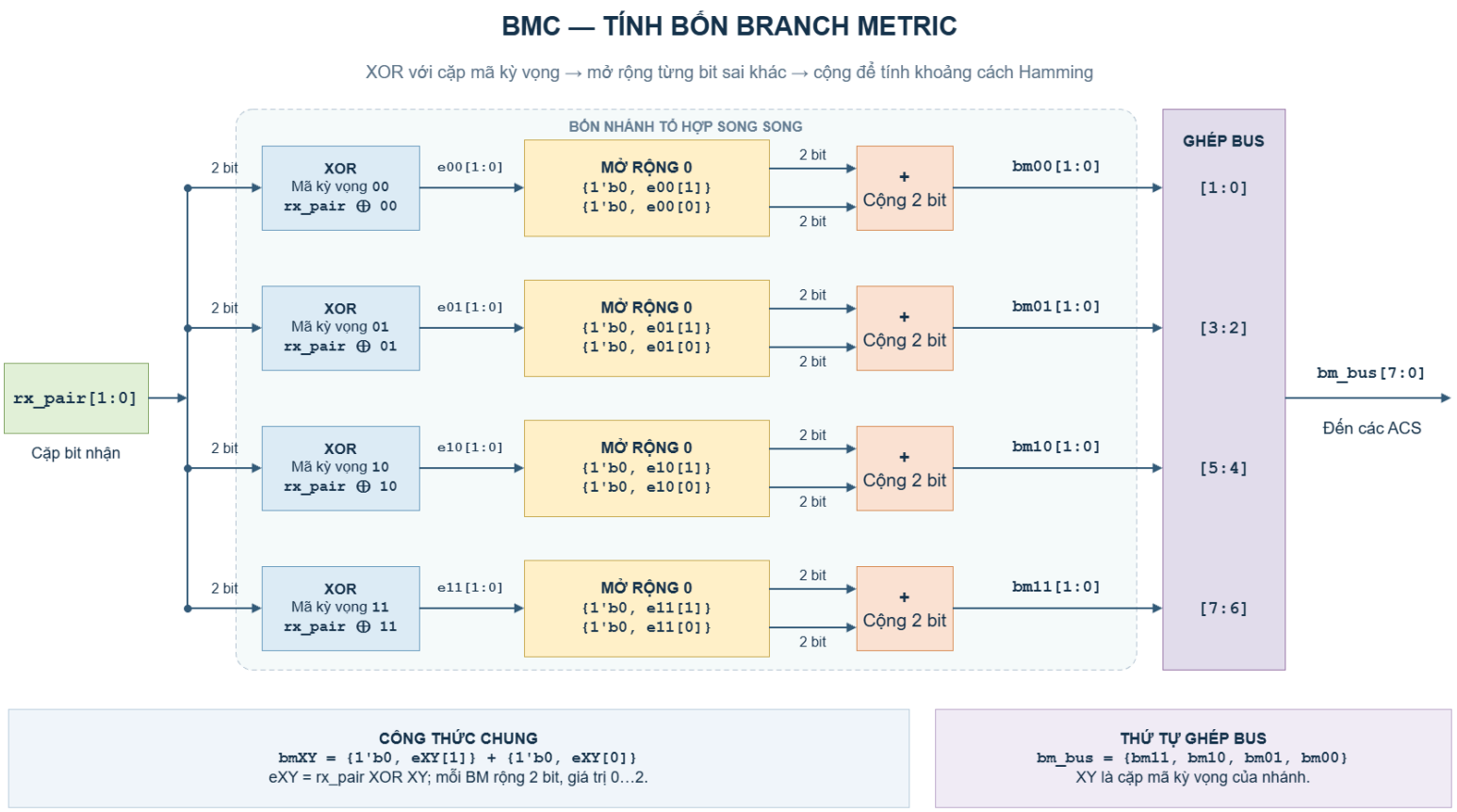
Logic tổ hợp hoàn toàn, không clock/reset/enable và không pipeline. Với cặp mã c1c0, tạo hai bit lỗi bằng XOR, mở rộng thành 2 bit rồi cộng. Đầu ra phải ổn định trước cạnh các thanh ghi PM chốt kết quả.

Bảng 19. Giá trị Branch Metric theo cặp bit nhận

rx_pair	bm00	bm01	bm10	bm11
00	0	1	1	2
01	1	0	2	1
10	1	2	0	1
11	2	1	1	0

Cấu trúc cho mỗi cặp mã: **rx_pair** XOR c1c0 → hai bit lỗi e1,e0 → BM = {0,e1} + {0,e0}. Bảng 5 tại mục 3.2 xác định cặp mã và BM cấp cho từng ngõ A/B của **ACS**. Bốn đầu vào nhị phân đều hợp lệ; X/Z là đầu vào không hợp lệ trong mô phỏng và phải được phát hiện tại giao diện kiểm thử. Cấu trúc BMC được minh họa tại Hình 11.

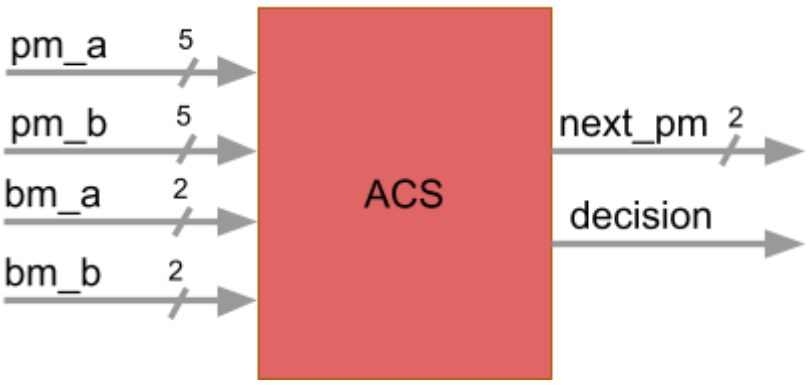
Hình 11. Sơ đồ logic BMC tính bốn Branch Metric



9.3 metric_update, ACS và thanh ghi PM

9.3.1 ACS

Hình 12. Giao diện module acs



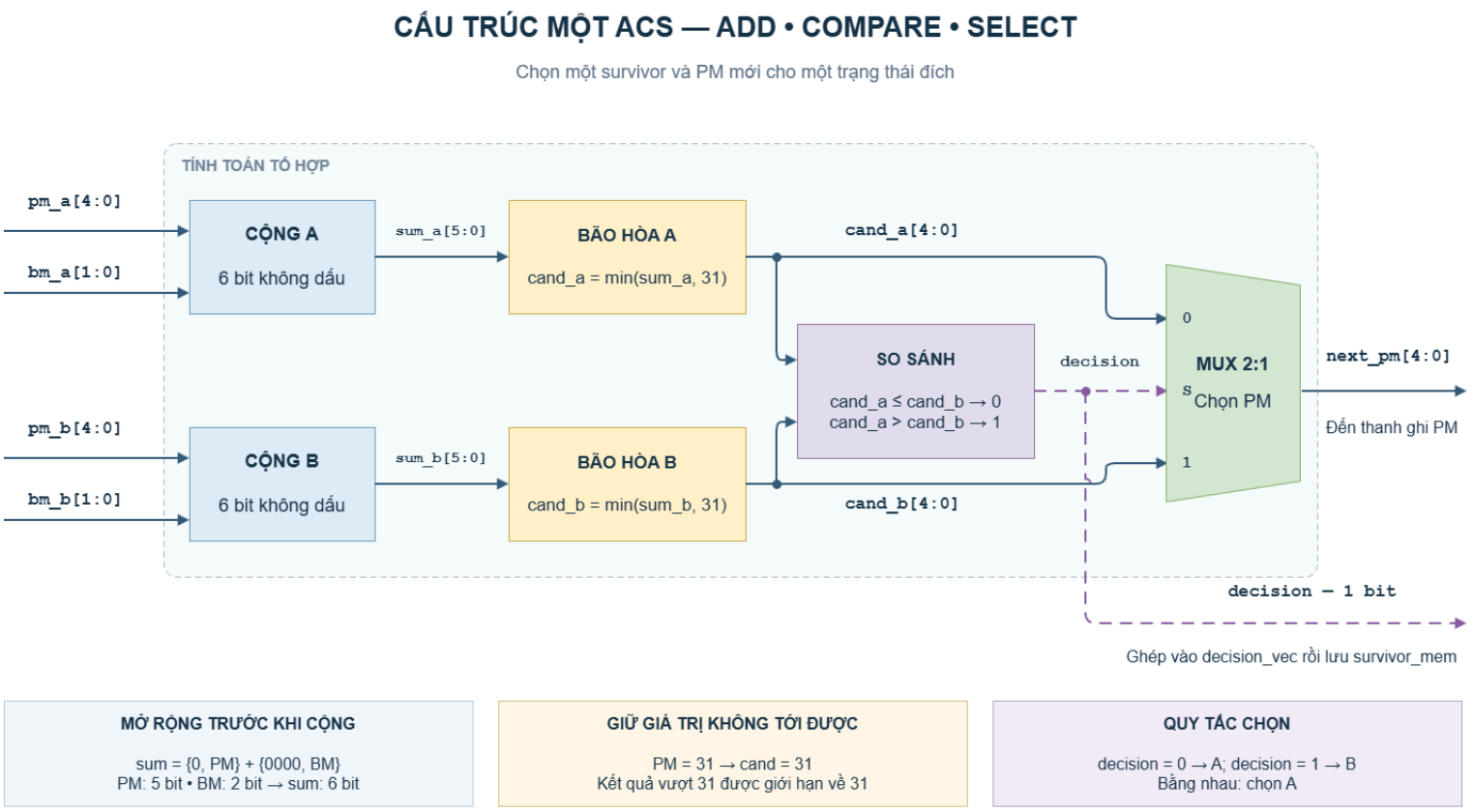
Bảng 20. Giao diện module `ACS`

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`pm_a`	Vào	5	PM cũ của trạng thái cha A; 31 biểu thị không tới được.
`pm_b`	Vào	5	PM cũ của trạng thái cha B; 31 biểu thị không tới được.
`bm_a`	Vào	2	BM của nhánh từ cha A đến trạng thái đích; giá trị 0…2.
`bm_b`	Vào	2	BM của nhánh từ cha B đến trạng thái đích; giá trị 0…2.
`next_pm`	Ra	5	PM nhỏ hơn trong hai ứng viên sau bão hòa về 31.
`decision`	Ra	1	0: chọn cha A; 1: chọn cha B. Hai ứng viên bằng nhau thì chọn A.

`ACS` là tổ hợp, tính bão hòa và ưu tiên A khi hòa theo mục 3.3.

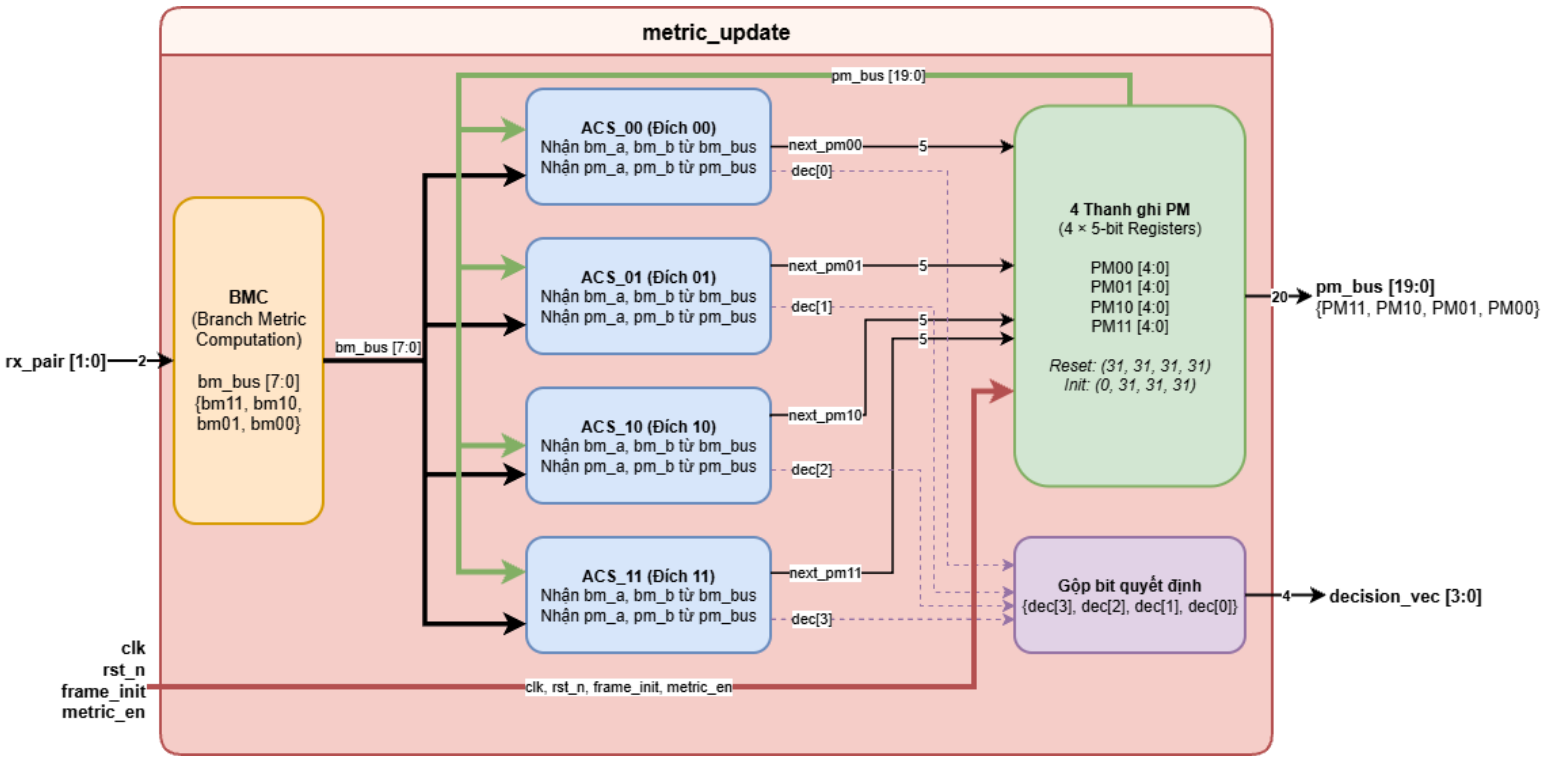
`BMC` dùng chung cho 4 `ACS`, ánh xạ cha/BM theo bảng trellis. Reset đặt cả 4 PM = 31; `frame_init` đặt (0,31,31,31); `metric_en` chốt đồng thời `next_pm`; không có enable thì giữ. `decision_vec` chỉ được ghi vào survivor khi `survivor_we` = 1. Hình 13 mô tả cấu trúc bên trong một ACS.

Hình 13. Cấu trúc một ACS với phép cộng, bão hòa, so sánh và chọn



9.3.2 metric_update và thanh ghi PM

Hình 14. Giao diện và cấu trúc module metric_update

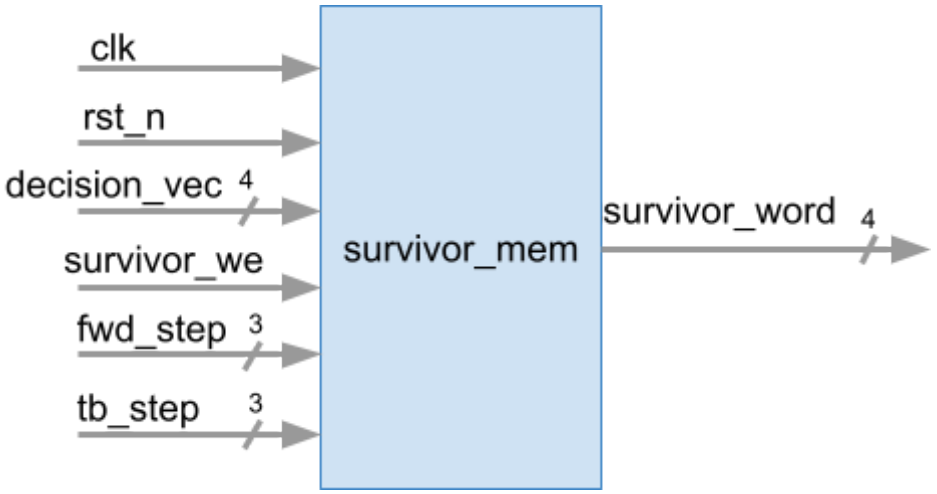


Bảng 21. Giao diện module `metric_update`

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`clk`	Vào	1	Clock; chốt đồng thời bốn PM tại cạnh lên.
`rst_n`	Vào	1	Reset bất đồng bộ, tích cực mức 0; đặt cả bốn PM bằng 31.
`frame_init`	Vào	1	Khởi tạo PM theo thứ tự 00, 01, 10, 11 thành (0,31,31,31); ưu tiên hơn `metric_en`.
`metric_en`	Vào	1	Cho phép cập nhật cả bốn PM từ kết quả `ACS`.
`rx_pair`	Vào	2	Cặp bit nhận cấp cho `BMC` dùng chung.
`pm_bus`	Ra	20	Bốn PM hiện tại, mỗi PM 5 bit; đóng gói theo mục 6.3.
`decision_vec`	Ra	4	Bốn `decision` bit tổ hợp từ PM cũ và `rx_pair`; mỗi bit ứng với một trạng thái đích.

9.4 `survivor_mem`

Hình 15. Giao diện module survivor_mem



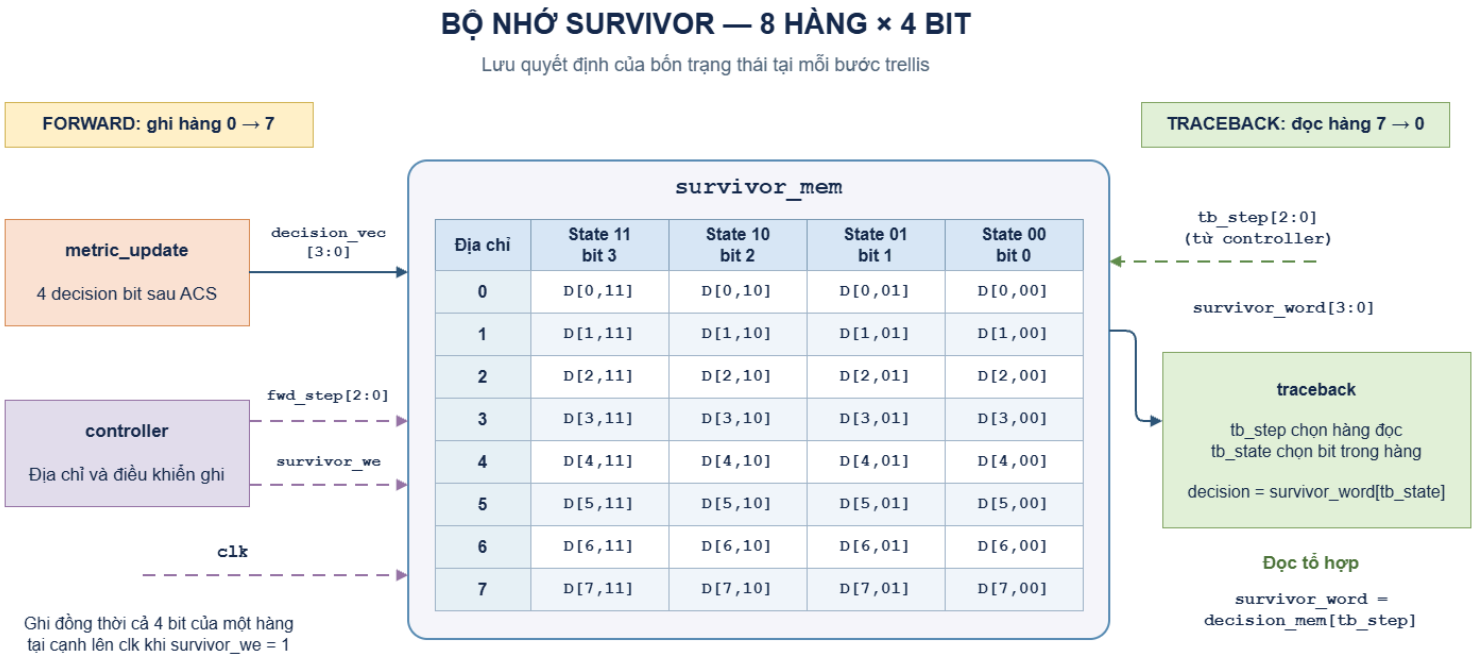
Bảng 22. Giao diện module `survivor_mem`

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`clk`	Vào	1	Clock; ghi bộ nhớ tại cạnh lên.
`rst_n`	Vào	1	Tích cực mức 0; khóa ghi khi reset, không xóa mảng **`survivor_mem`**.
`survivor_we`	Vào	1	Cho phép ghi **`decision_vec`** khi **`rst_n`** = 1.
`fwd_step`	Vào	3	Địa chỉ hàng ghi 0…7; hàng j lưu bước trellis t = j + 1.
`decision_vec`	Vào	4	Bốn **`decision`** bit được ghi đồng thời vào hàng đã chọn.
`tb_step`	Vào	3	Địa chỉ hàng đọc; chạy từ 7 về 0 khi truy vết.
`survivor_word`	Ra	4	Đầu ra đọc tổ hợp của **`survivor_mem`**[**`tb_step`**].

Tại cạnh lên, nếu **`rst_n`** = 1 và **`survivor_we`** = 1: **`survivor_mem`**[**`fwd_step`**] nhận **`decision_vec`**. Đầu ra tổ hợp = **`survivor_mem`**[**`tb_step`**].

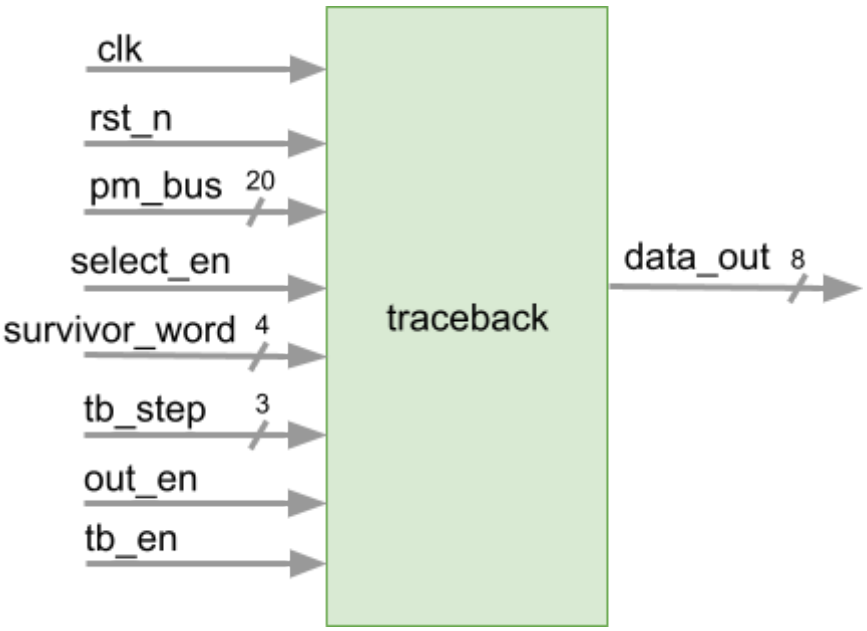
Không reset mảng 32 bit. Controller bảo đảm ghi hàng 0…7 trước khi bật **`tb_en`** và không ghi trong SELECT/TRACEBACK/DONE. Giá trị đọc trước khi hoàn tất forward không được dùng để sinh kết quả. Tổ chức bộ nhớ và địa chỉ ghi, đọc được minh họa tại Hình 16.

Hình 16. Tổ chức bộ nhớ survivor 8 × 4 bit và địa chỉ ghi, đọc



9.5 `traceback` và thanh ghi đầu ra

Hình 17. Giao diện module traceback



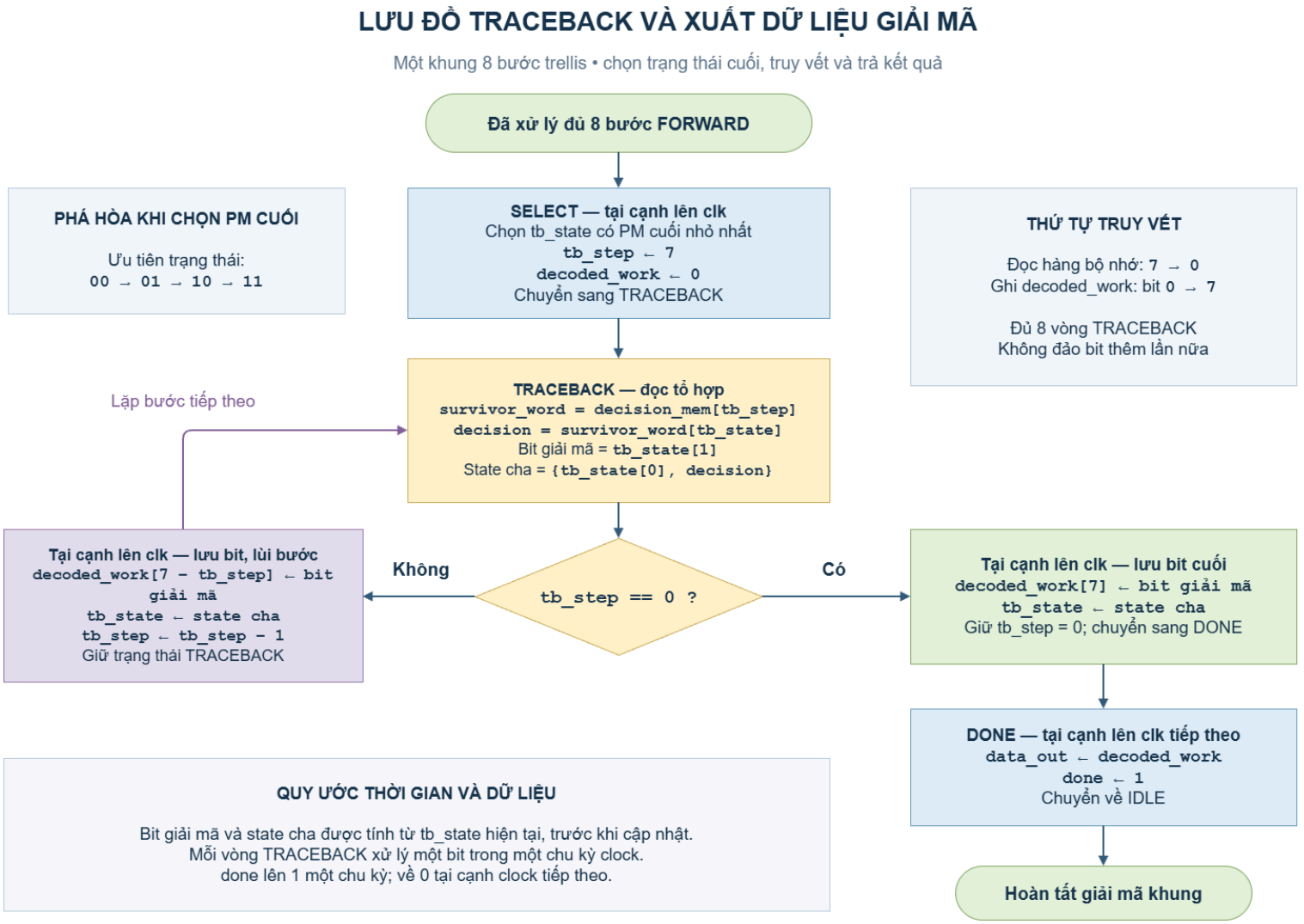
Bảng 23. Giao diện module `traceback`

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`clk`	Vào	1	Clock; cập nhật các thanh ghi truy vết và đầu ra tại cạnh lên.
`rst_n`	Vào	1	Reset bất đồng bộ, tích cực mức 0; xóa **`tb_state`**, **`decoded_work`** và **`data_out`**.
`pm_bus`	Vào	20	Bốn PM cuối để chọn trạng thái bắt đầu truy vết.
`survivor_word`	Vào	4	Hàng **`decision`** đang đọc; dùng **`tb_state`** chọn một bit.
`tb_step`	Vào	3	Chỉ số bước truy vết 7…0; vị trí ghi là **`decoded_work`**[7 − **`tb_step`**].
`select_en`	Vào	1	Chọn trạng thái có PM nhỏ nhất và xóa **`decoded_work`**.
`tb_en`	Vào	1	Thực hiện một bước truy vết và ghi một bit vào **`decoded_work`**.
`out_en`	Vào	1	Chuyển đủ 8 bit **`decoded_work`** sang **`data_out`**.
`data_out`	Ra	8	Kết quả giải mã; giữ giá trị đến lần xuất tiếp theo hoặc reset.

Reset xóa **`tb_state`**, **`decoded_work`** và **`data_out`**. **`select_en`** chốt state có PM nhỏ nhất và xóa **`decoded_work`**; **`tb_en`** thực hiện đúng một bước theo mục 3.4; **`out_en`** chuyển **`decoded_work`** sang **`data_out`**.

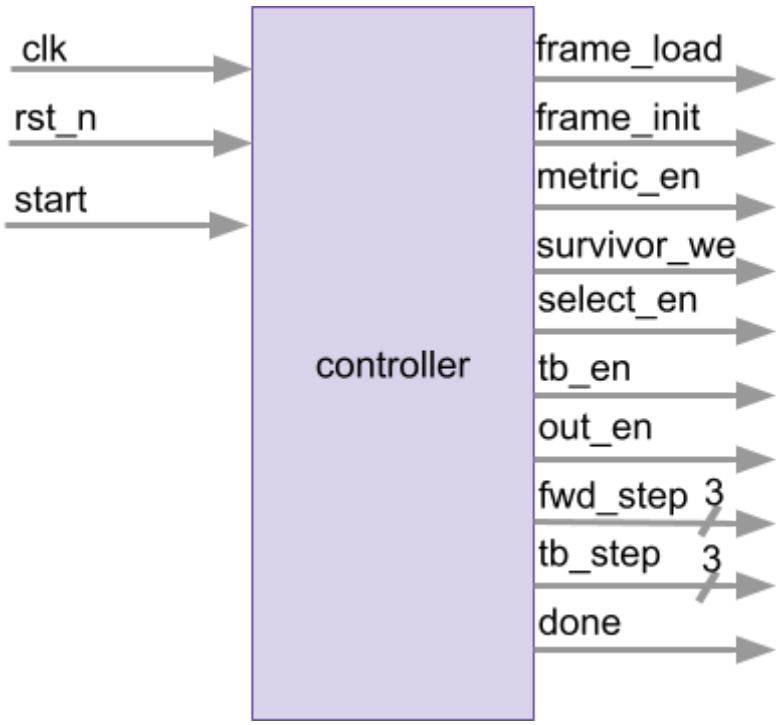
Ưu tiên reset → **`select_en`** → **`tb_en`** → **`out_en`**; **`controller`** chỉ phát tối đa một trong ba enable. Khi không có enable, giữ giá trị. Lần ghi **`decoded_work[7]`** tại E17 hoàn thành trước khi xuất cả thanh ghi tại E18. Các bước truy vết và xuất kết quả được tổng hợp tại Hình 18.

Hình 18. Lưu đồ truy vết và xuất dữ liệu giải mã



9.6 `controller`

Hình 19. Giao diện module controller



Bảng 24. Giao diện module `controller`

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`clk`	Vào	1	Clock; cập nhật FSM, bộ đếm và **`done`** tại cạnh lên.
`rst_n`	Vào	1	Reset bất đồng bộ, tích cực mức 0; về IDLE, xóa bộ đếm và **`done`**.
`start`	Vào	1	Yêu cầu nhận khung; chỉ được chấp nhận ở IDLE.
`frame_load`	Ra	1	IDLE và **`start`** = 1: chốt khung vào **`input_buffer`**.
`frame_init`	Ra	1	IDLE và **`start`** = 1: khởi tạo PM cho khung mới.
`metric_en`	Ra	1	FORWARD: cho phép cập nhật bốn PM.
`survivor_we`	Ra	1	FORWARD: ghi bốn **`decision`** bit cùng cạnh cập nhật PM.
`select_en`	Ra	1	SELECT: chọn trạng thái bắt đầu truy vết và xóa **`decoded_work`**.

Tên tín hiệu	Chiều	Độ rộng (bit)	Ý nghĩa
`tb_en`	Ra	1	TRACEBACK: cho phép thực hiện một bước truy vết.
`out_en`	Ra	1	DONE: cho phép cập nhật **`data_out`**.
`fwd_step`	Ra	3	Chỉ số FORWARD 0…7; giữ tại 7 sau bước cuối.
`tb_step`	Ra	3	Nạp 7 tại SELECT; giảm trong TRACEBACK và giữ tại 0 sau bước cuối.
`done`	Ra	1	Xung một clock báo kết quả hợp lệ; lên 1 sau cạnh xử lý DONE.

FSM và bộ đếm theo mục 7; enable được giải mã từ trạng thái hiện tại và **`start`**, giữ mặc định 0 ngoài điều kiện quy định. Không nhận yêu cầu khi bận và không lưu hàng đợi.